

# Digital Engineering

## Simplification of Boolean Functions

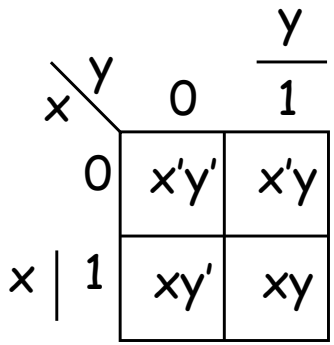
# Overview

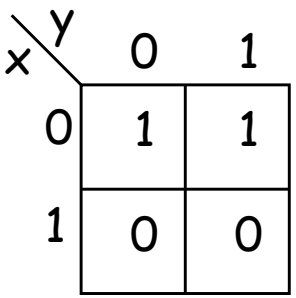
- Introduction to Karnaugh Maps
- Karnaugh Maps Rules and Methods
- Two and Three-Variable K-Maps
- Four-Variable K-Maps
- Simplification Techniques
- Don’t Cares Conditions

# Karnaugh Maps

- Alternate way of representing Boolean functions
  - Each row in the truth table is represented by a square
  - Each square represents a *minterm*
- Easy to convert between truth table, K-map, and SOP
  - Un-optimized form: number of 1's in K-map equals number of minterms (products) in SOP
  - Optimized form: reduced number of minterms

$$F = \Sigma(m_0, m_1) = x'y' + x'y$$




| x | y | F |
|---|---|---|
| 0 | 0 | 1 |
| 0 | 1 | 1 |
| 1 | 0 | 0 |
| 1 | 1 | 0 |

# Karnaugh Maps (cont'd)

- A Karnaugh map is a graphical tool for assisting in the general simplification procedure
- Two variable maps

| A \ B | 0 | 1 |
|---|---|---|
| 0 | 0 | 1 |
| 1 | 1 | 0 |

$F=A\overline{B}+\overline{A}B$

| A \ B | 0 | 1 |
|---|---|---|
| 0 | 0 | 1 |
| 1 | 1 | 1 |

$F=AB+\overline{A}B+A\overline{B}$

- Three variable maps

| A \ BC | 00 | 01 | 11 | 10 |
|---|---|---|---|---|
| 0 | 0 | 1 | 0 | 1 |
| 1 | 1 | 1 | 1 | 1 |

| A | B | C | F |
|---|---|---|---|
| 0 | 0 | 0 | 0 |
| 0 | 0 | 1 | 1 |
| 0 | 1 | 0 | 1 |
| 0 | 1 | 1 | 0 |
| 1 | 0 | 0 | 1 |
| 1 | 0 | 1 | 1 |
| 1 | 1 | 0 | 1 |
| 1 | 1 | 1 | 1 |

$$F=A\overline{B}\,\overline{C}+A\overline{B}C+ABC+AB\overline{C}+\overline{A}\overline{B}C+\overline{A}B\overline{C}$$

# Rules for K-Maps

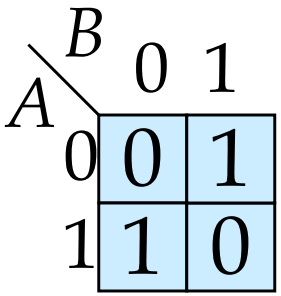
- We can reduce functions by circling 1's in the K-map
- Each circle represents minterm reduction
- After circling, deduce a minimized AND-OR form

*Rules to consider*

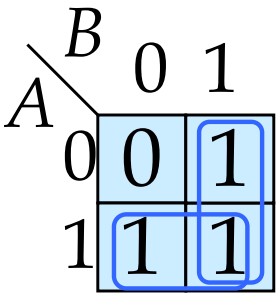
- Every cell containing a 1 must be included at least once
- The largest possible "power of 2 rectangle" should be used
- Use the smallest possible number of rectangles

# Karnaugh Maps Examples

- Two variable maps



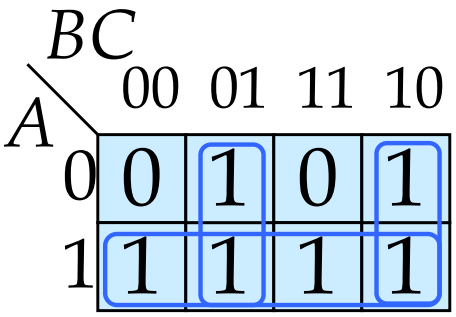

$F=A\overline{B}+\overline{A}B$




$F=AB+\overline{A}B+A\overline{B}$

$F=A+B$

- Three variable maps

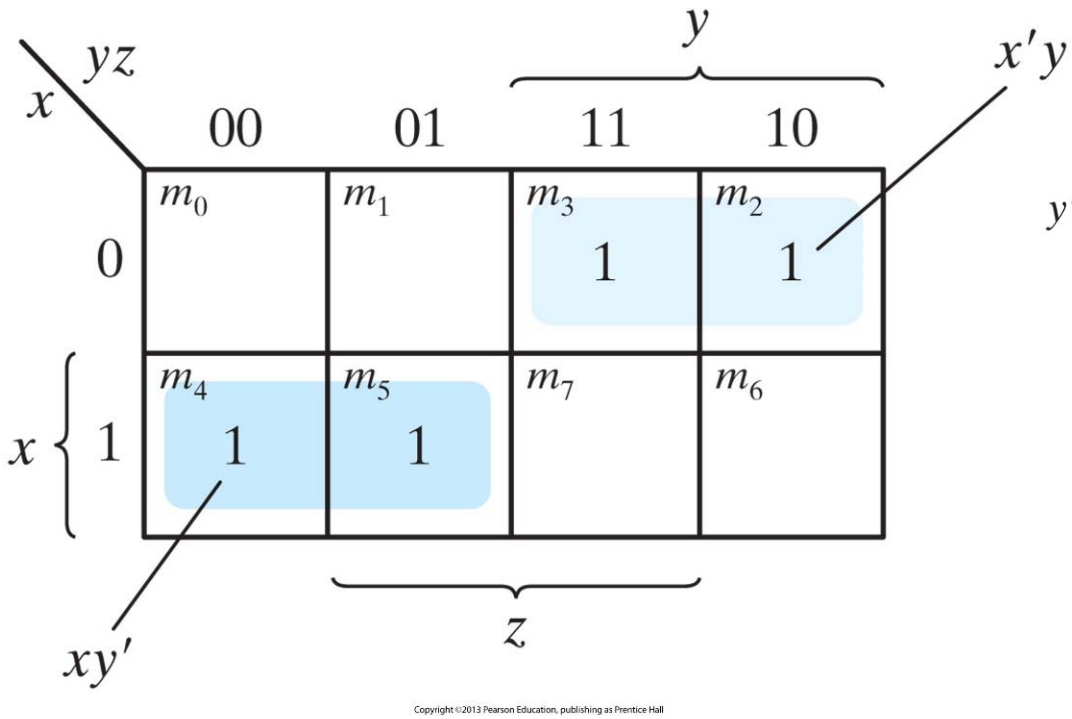



$F=A+\overline{B}C+B\overline{C}$

$$F=A\overline{B}\,\overline{C}+A\overline{B}C+ABC+AB\overline{C}+\overline{A}\,\overline{B}C+\overline{A}B\overline{C}$$

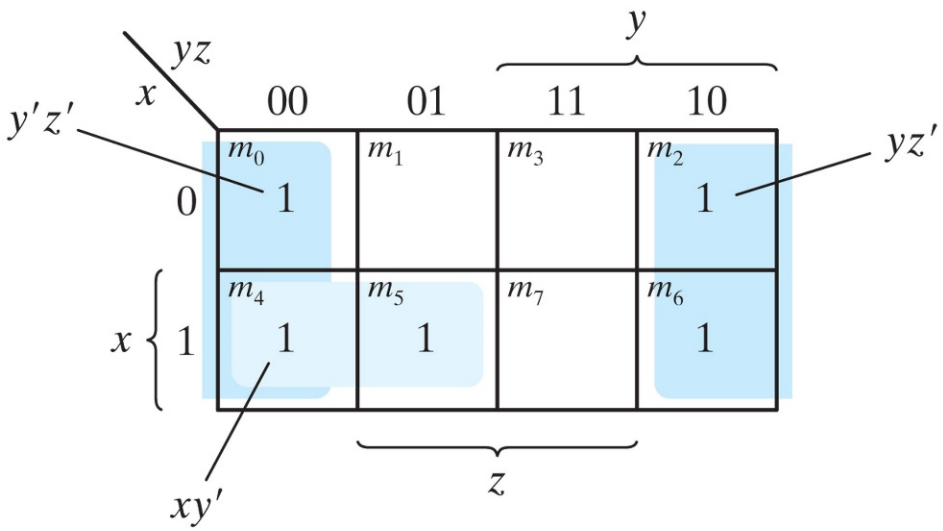
# Karnaugh Maps Examples (cont'd)

**FIGURE 3.4** Map for Example 3.1, $F(x, y, z) = \Sigma(2, 3, 4, 5) = x'y + xy'$



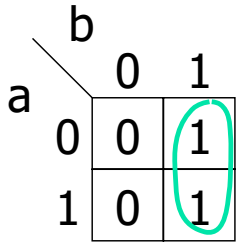

Copyright ©2013 Pearson Education, publishing as Prentice Hall

**FIGURE 3.6** Map for Example 3.3, $F(x, y, z) = \Sigma(0, 2, 4, 5, 6) = z' + xy'$

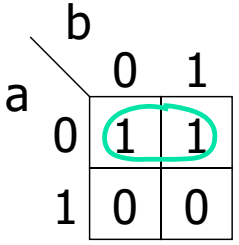



Copyright ©2013 Pearson Education, publishing as Prentice Hall

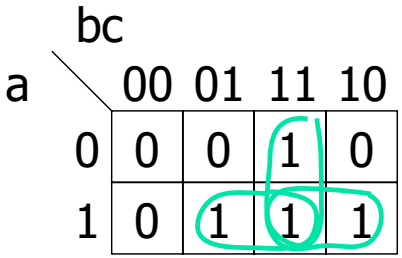
# Karnaugh Maps Examples (cont'd)



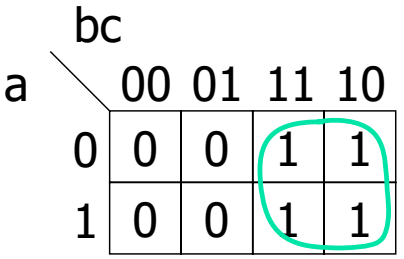

f = b




g = a'




cout = bc + ac + ab

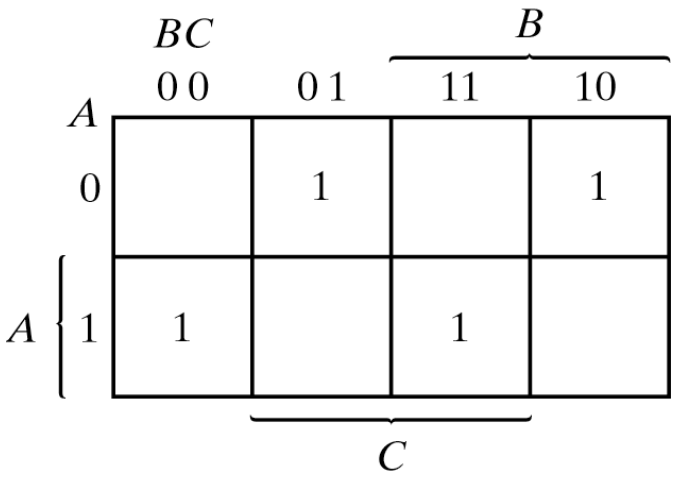



f = b

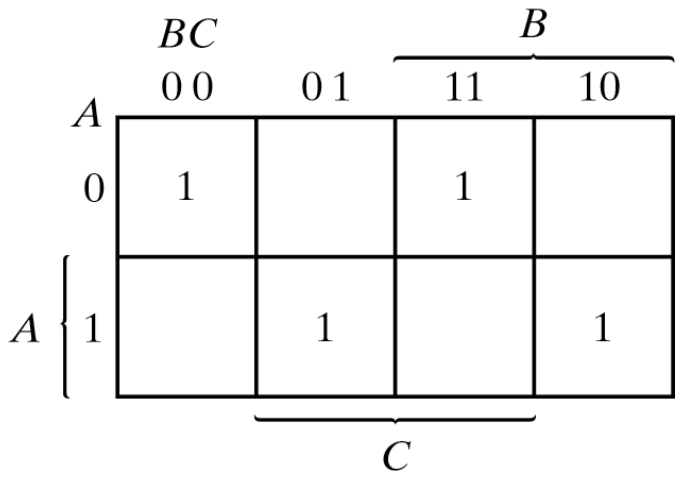
1. Circle the largest groups possible
2. Group dimensions must be a power of 2
3. Remember what circling means!

# K-Maps of 3-variable XOR Function

- Odd function – odd number of inputs are 1
- Even function – even number of inputs are 1



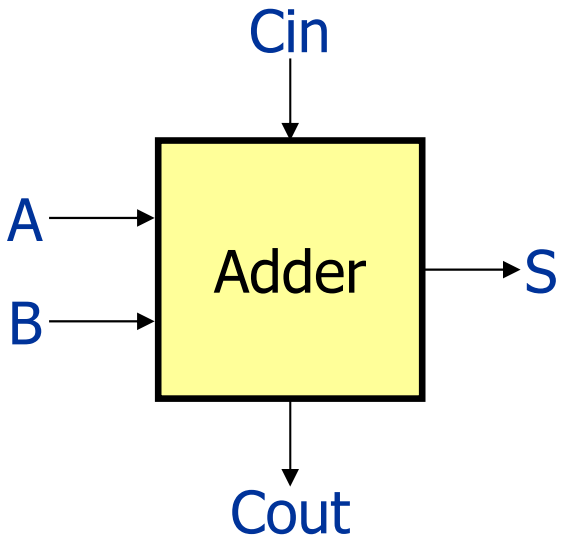

(a) Odd function
$F = A \oplus B \oplus C$




(a) Even function
$F = (A \oplus B \oplus C)'$

Fig. 3-33 Map for a Three-variable Exclusive-OR Function

# Example using 1-bit Adder




| A | B | Cin | S | Cout |
|---|---|---|---|---|
| 0 | 0 | 0 | 0 | 0 |
| 0 | 0 | 1 | 1 | 0 |
| 0 | 1 | 0 | 1 | 0 |
| 0 | 1 | 1 | 0 | 1 |
| 1 | 0 | 0 | 1 | 0 |
| 1 | 0 | 1 | 0 | 1 |
| 1 | 1 | 0 | 0 | 1 |
| 1 | 1 | 1 | 1 | 1 |

How to use a Karnaugh Map instead of the Algebraic simplification?

$$S = A'B'Cin + A'BCin' + A'BCin + ABCin$$

$$Cout = A'BCin + A\,B'Cin + ABCin' + ABCin$$

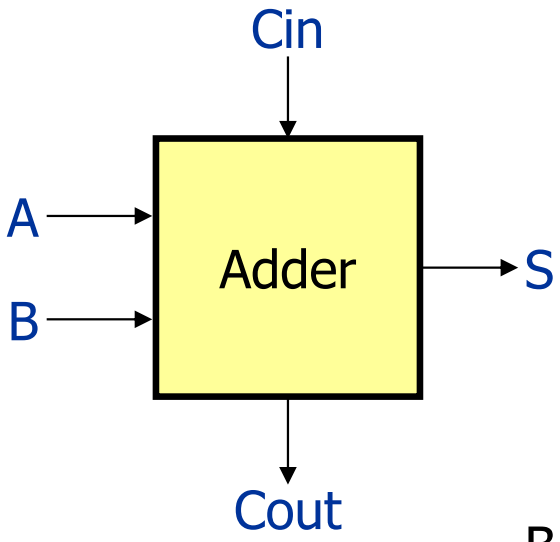
$$= A'BCin + ABCin + AB'Cin + ABCin + ABCin' + ABCin$$

$$= (A' + A)BCin + (B' + B)ACin + (Cin' + Cin)AB$$

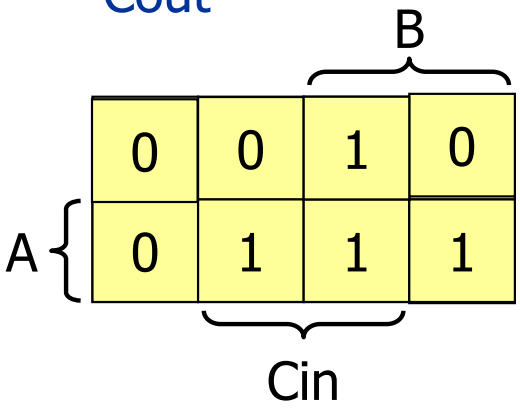
$$= 1 \cdot BCin + 1 \cdot ACin + 1 \cdot AB$$

$$= BCin + ACin + AB$$

# Example using 1-bit Adder (cont'd)



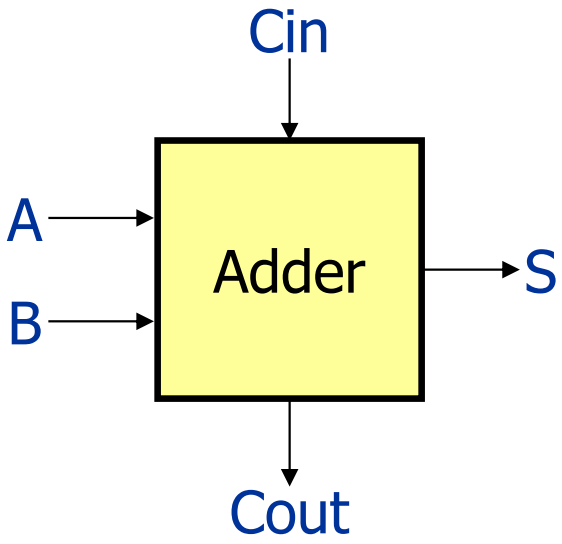

| A | B | Cin | S | Cout |
|---|---|---|---|---|
| 0 | 0 | 0 | 0 | 0 |
| 0 | 0 | 1 | 1 | 0 |
| 0 | 1 | 0 | 1 | 0 |
| 0 | 1 | 1 | 0 | 1 |
| 1 | 0 | 0 | 1 | 0 |
| 1 | 0 | 1 | 0 | 1 |
| 1 | 1 | 0 | 0 | 1 |
| 1 | 1 | 1 | 1 | 1 |



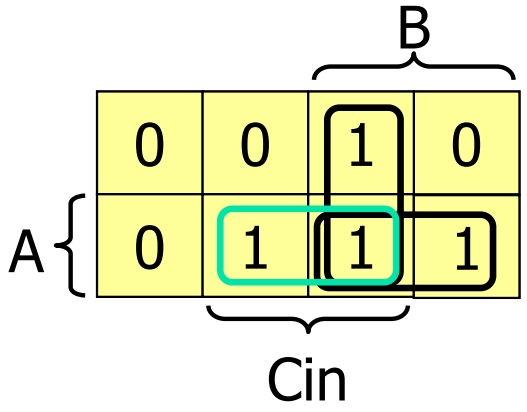

Karnaugh Map for Cout

Now we have to cover all the 1s in the Karnaugh Map using the largest rectangles and as few rectangles as we can.

# Example using 1-bit Adder (cont'd)




| A | B | Cin | S | Cout |
|---|---|---|---|---|
| 0 | 0 | 0 | 0 | 0 |
| 0 | 0 | 1 | 1 | 0 |
| 0 | 1 | 0 | 1 | 0 |
| 0 | 1 | 1 | 0 | 1 |
| 1 | 0 | 0 | 1 | 0 |
| 1 | 0 | 1 | 0 | 1 |
| 1 | 1 | 0 | 0 | 1 |
| 1 | 1 | 1 | 1 | 1 |

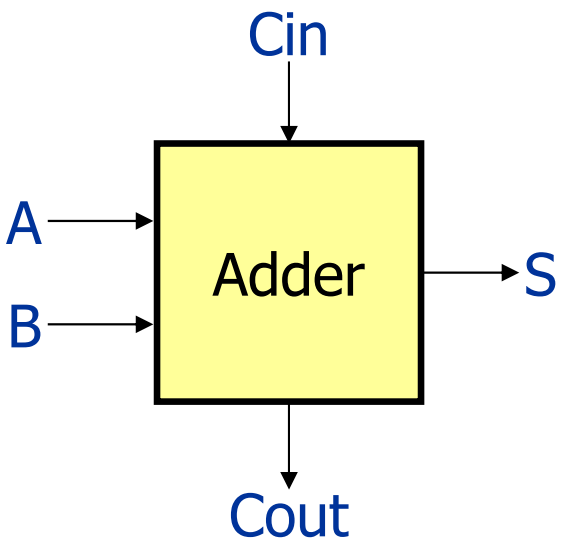



Karnaugh Map for Cout

Now we have to cover all the 1s in the Karnaugh Map using the largest rectangles and as few rectangles as we can.

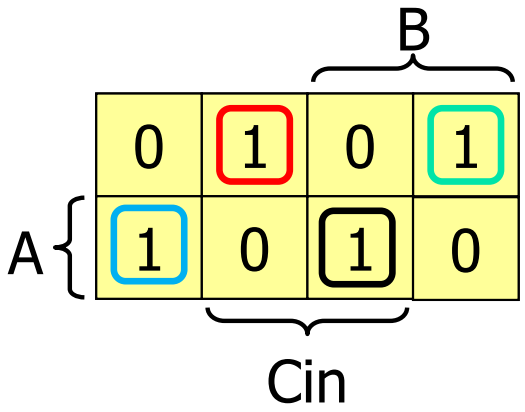
$$Cout = BCin + AB + ACin$$

# Example using 1-bit Adder (cont'd)




Can you draw the circuit diagrams?

| A | B | Cin | S | Cout |
|---|---|---|---|---|
| 0 | 0 | 0 | 0 | 0 |
| 0 | 0 | 1 | 1 | 0 |
| 0 | 1 | 0 | 1 | 0 |
| 0 | 1 | 1 | 0 | 1 |
| 1 | 0 | 0 | 1 | 0 |
| 1 | 0 | 1 | 0 | 1 |
| 1 | 1 | 0 | 0 | 1 |
| 1 | 1 | 1 | 1 | 1 |




S = A′ B′ Cin + A′BC′in + A B′ C′in + ABCin

No Possible Reduction!

# Karnaugh Maps for 4-Input Functions

- Represent functions of 4 inputs with 16 minterms
- Use same rules developed for 3-input functions

| | | | |
|---|---|---|---|
| $m_0$ | $m_1$ | $m_3$ | $m_2$ |
| $m_4$ | $m_5$ | $m_7$ | $m_6$ |
| $m_{12}$ | $m_{13}$ | $m_{15}$ | $m_{14}$ |
| $m_8$ | $m_9$ | $m_{11}$ | $m_{10}$ |

(a)

| $wx$ \ $yz$ | 00 | 01 | 11 | 10 |
|---|---|---|---|---|
| 00 | $w'x'y'z'$ | $w'x'y'z$ | $w'x'yz$ | $w'x'yz'$ |
| 01 | $w'xy'z'$ | $w'xy'z$ | $w'xyz$ | $w'xyz'$ |
| 11 | $wxy'z'$ | $wxy'z$ | $wxyz$ | $wxyz'$ |
| 10 | $wx'y'z'$ | $wx'y'z$ | $wx'yz$ | $wx'yz'$ |

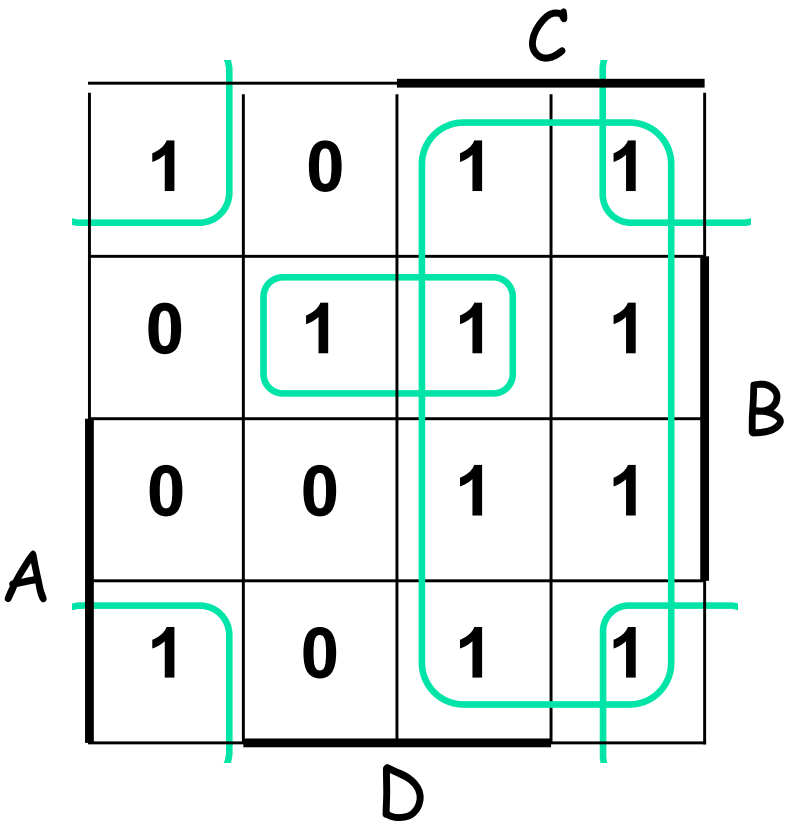
y (11, 10); x (01, 11); w (11, 10); z (00, 01)

(b)

Fig. 3-8 Four-variable Map

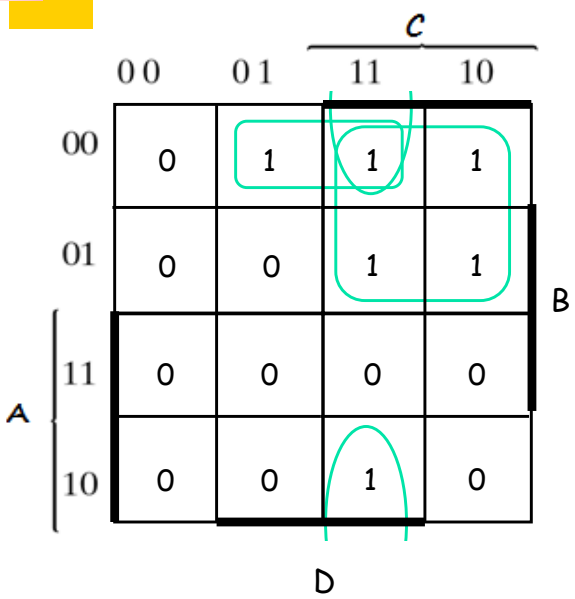
# Karnaugh Map: 4-Variable Example

$F(A,B,C,D) = \Sigma m(0, 2, 3, 5, 6, 7, 8, 10, 11, 14, 15)$

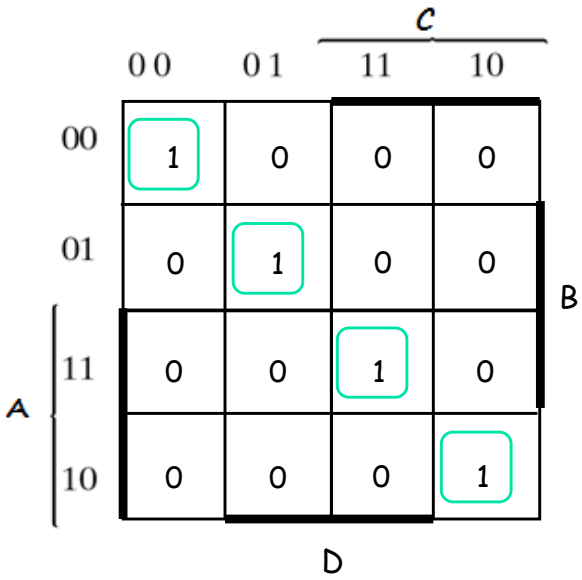



$F = C + A'BD + B'D'$

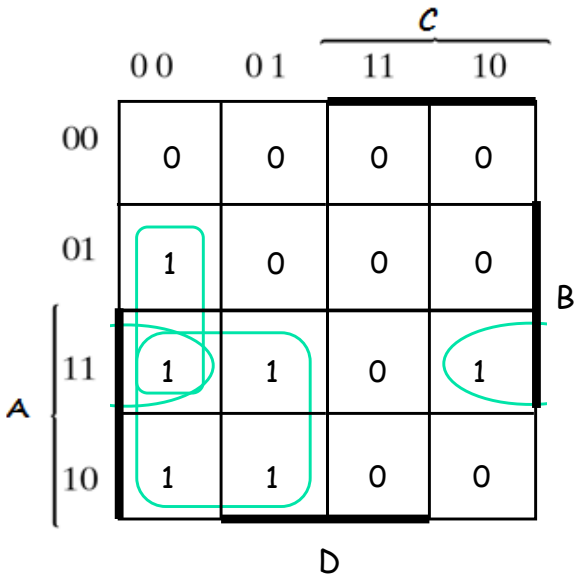
# Design Examples




K-map for LT




K-map for EQ




K-map for GT

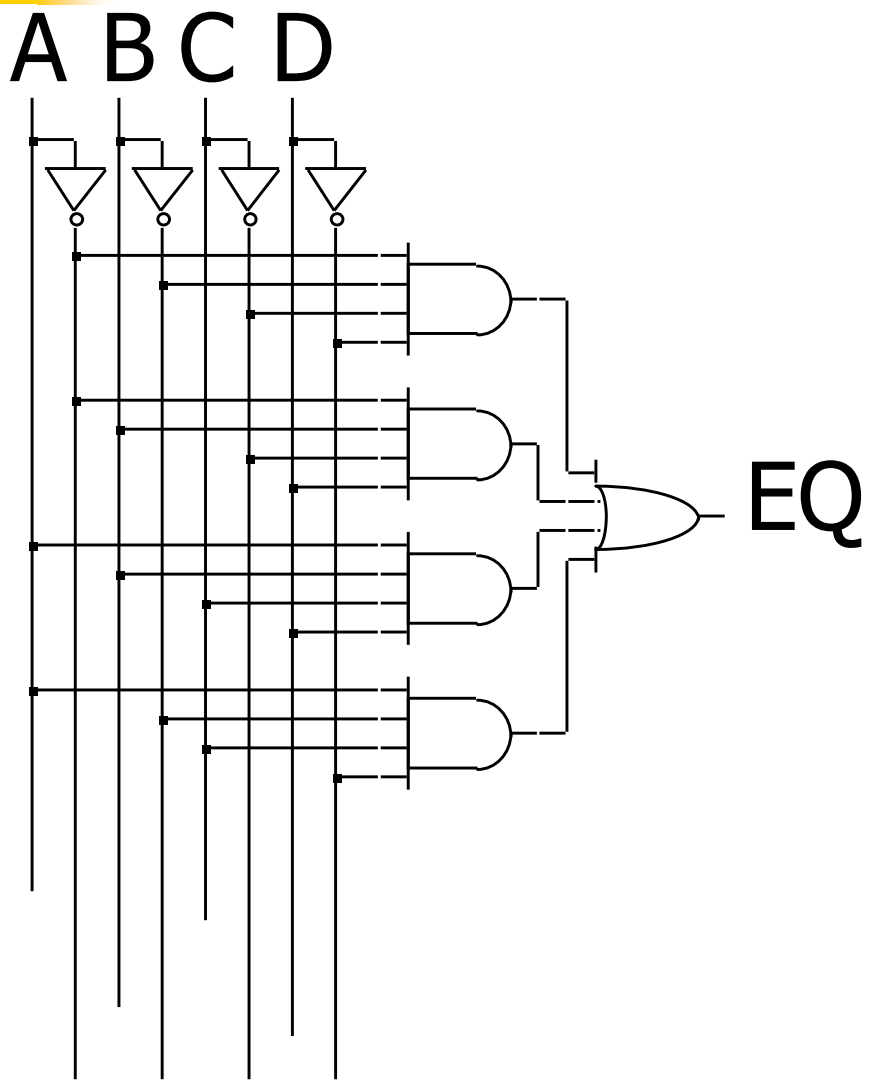
LT = A' C + A' B' D + B' C D

EQ = A'B'C'D' + A'BC'D + ABCD + AB'CD'

GT = A C' + B C' D' + A B D'

Can you draw the truth table for these examples?

# Physical Implementation

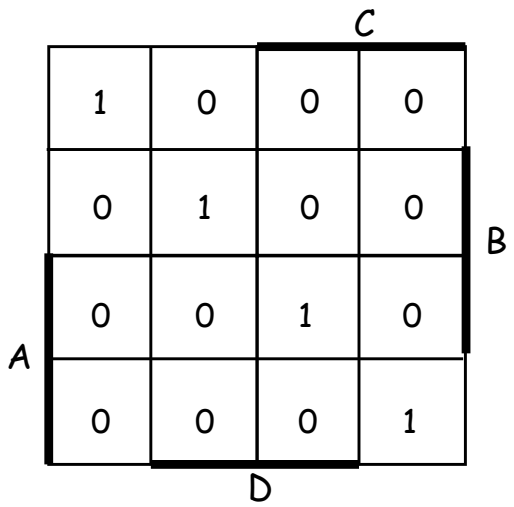



Step 1: Truth table

Step 2: K-map

Step 3: Minimized sum-of-products

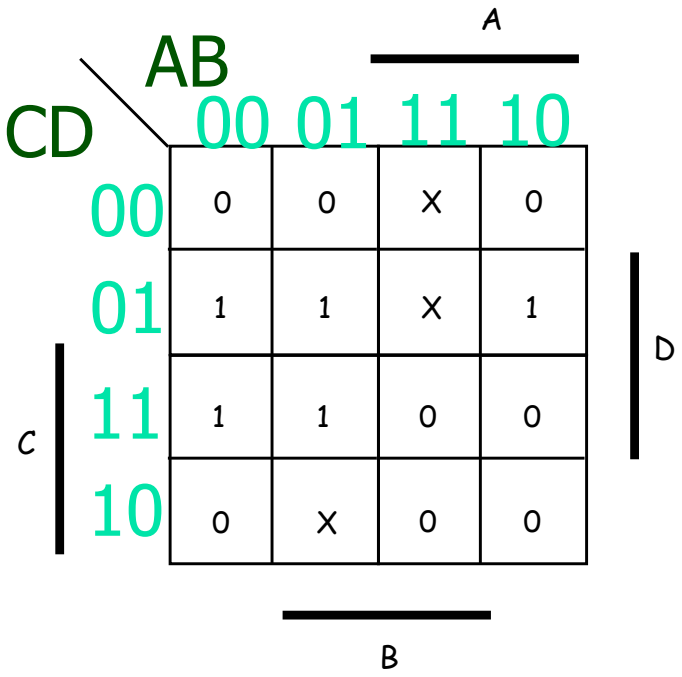
Step 4: Physical implementation with gates




K-map for EQ

# Karnaugh Maps: Don't Cares

- In some cases, outputs are undefined
- We "don't care" if the circuit produces a '0' or a '1'
- This knowledge can be used to simplify functions

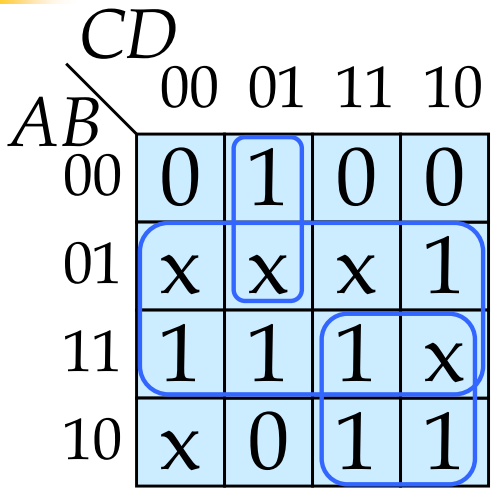



- Treat X's like either 1's or 0's
- Very useful
- OK to leave some X's uncovered

# Don’t Care Conditions

- In some situations, we don’t care about the value of a function for certain combinations of the variables.
  - these combinations may be impossible in certain contexts
  - or the value of  the function may not matter when the combinations occur
- In such situations we say the function is incompletely specified and there are multiple (completely specified) logic functions that can be used in the design.
  - so we can select a function that gives the simplest circuit
- When constructing the terms in the simplification procedure, we can choose to either cover or not cover the don’t care conditions.

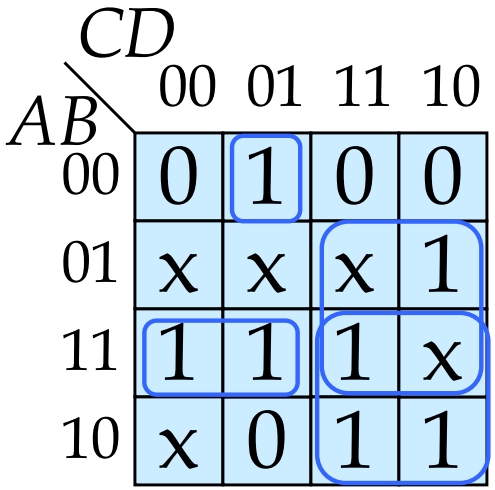
# Don’t Cares Examples



| AB \ CD | 00 | 01 | 11 | 10 |
|---|---|---|---|---|
| 00 | 0 | 1 | 0 | 0 |
| 01 | x | x | x | 1 |
| 11 | 1 | 1 | 1 | x |
| 10 | x | 0 | 1 | 1 |

$F=A'C'D+B+AC$

**Alternative covering:**



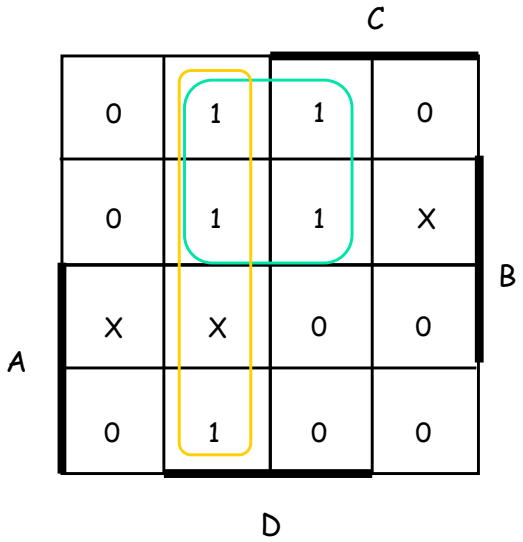
| AB \ CD | 00 | 01 | 11 | 10 |
|---|---|---|---|---|
| 00 | 0 | 1 | 0 | 0 |
| 01 | x | x | x | 1 |
| 11 | 1 | 1 | 1 | x |
| 10 | x | 0 | 1 | 1 |

$F=A'B'C'D+ABC'+BC+AC$

# Don’t Cares Examples (cont’d)

$f(A,B,C,D) = \Sigma\, m(1,3,5,7,9) + d(6,12,13)$

- $f = A'D + B'C'D$ without don't cares
- $f = A' D + C' D$ with don't cares

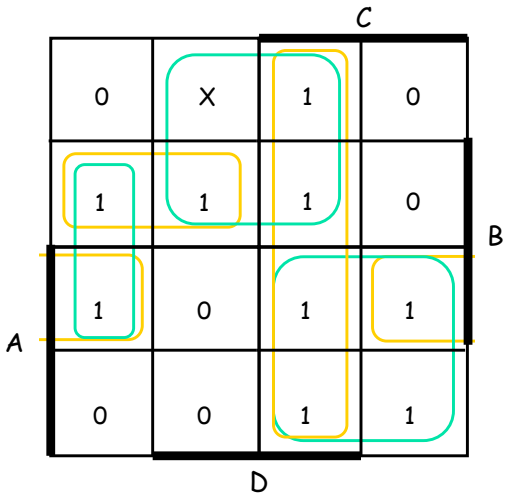



don't cares can be treated as 1s or 0s depending on which is more advantageous

# Definition of Terms

- Implicant
  - Single product term of the ON-set (terms that create a logic 1)
- Prime implicant
  - Implicant that can't be combined with another to form an implicant with fewer literals
- Essential prime implicant
  - Prime implicant is essential if it alone covers a minterm in the K-map
  - Remember that all squares marked with 1 must be covered
- Objective:
  - Group implicants into prime implicants (minimize literals per term)
  - Cover the K-map with as few prime implicants as possible (minimize number of product terms)

# Examples to Illustrate Terms




6 prime implicants:

ABD', A'D, AC, A'BC', CD, BC'D'

essential

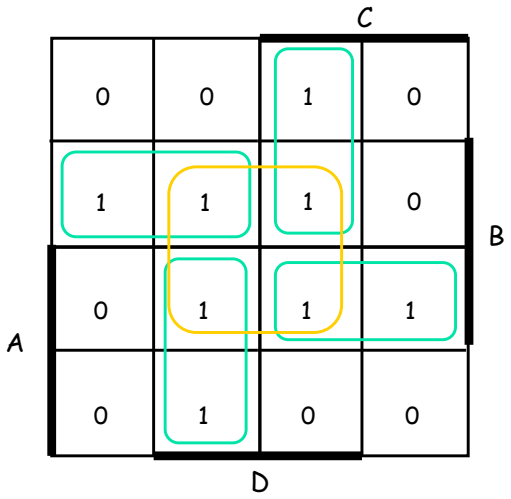
minimum cover: AC + A'D + BC'D'

5 prime implicants:

BD, ABC, AC'D, A'BC', A'CD

essential

minimum cover: 4 essential implicants

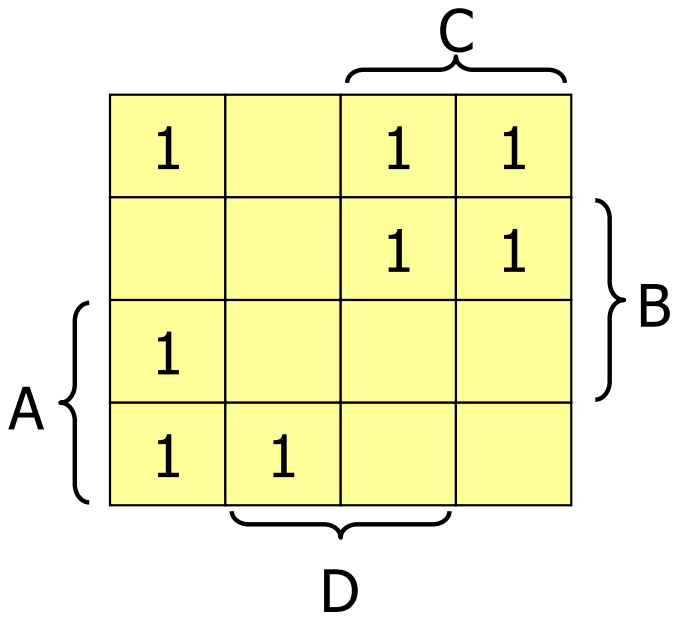


# Prime Implicants

Any single 1 or group of 1s in the Karnaugh map of a function F is an implicant of F.

A product term is called a prime implicant of F if it cannot be combined with another term to eliminate a variable.

Example:




If a function F is represented by this Karnaugh Map. Which of the following terms are implicants of F, and which ones are prime implicants of F?

(a) A'B'C
(b) BD
(c) A'B'C'D'
(d) A'C
(e) A'B'D'

Implicants:
(a),(c),(d),(e)

Prime Implicants:
(d),(e)

# Essential Prime Implicants

A product term is an essential prime implicant if there is a minterm that is only covered by that prime implicant

The minimal sum-of-products form of F must include all the essential prime implicants of F

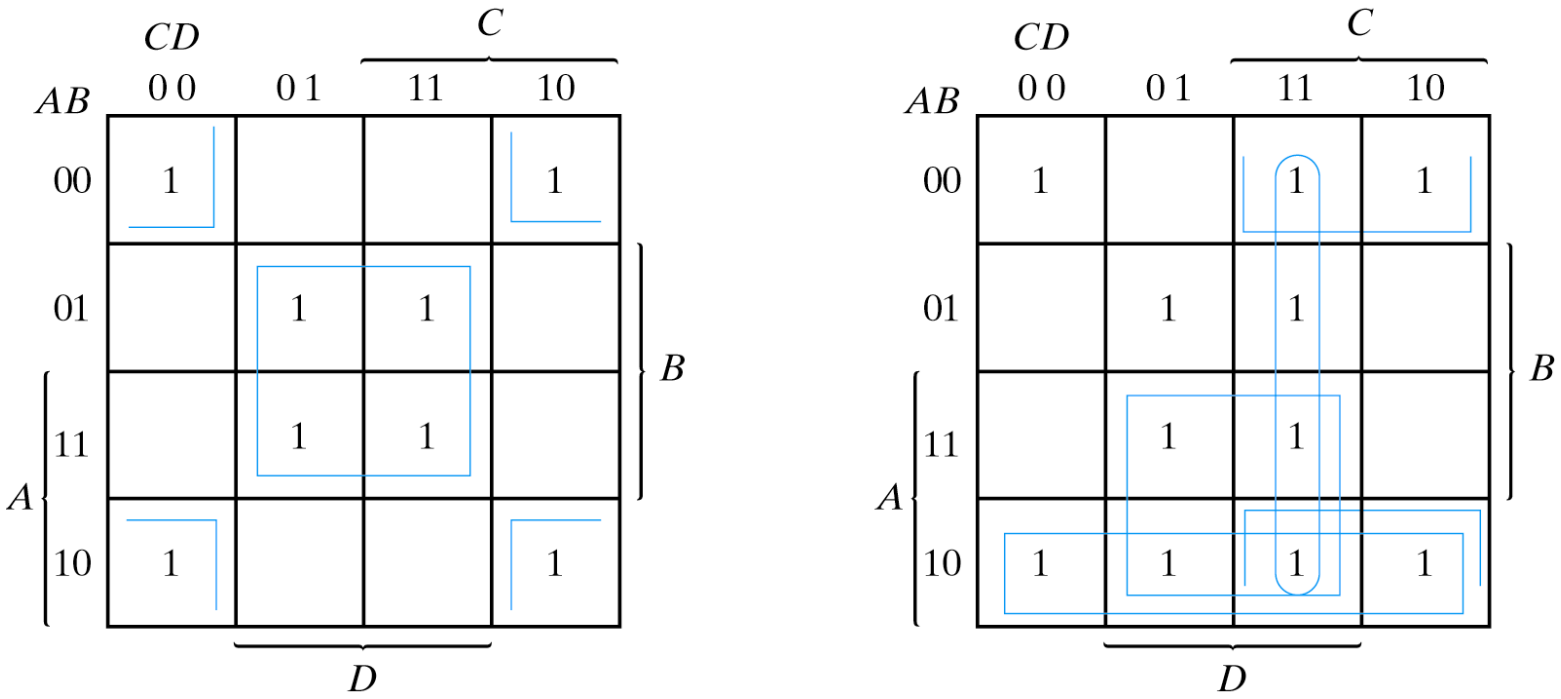



Fig. 3-11 Simplification Using Prime Implicants



# Summary

- Karnaugh map allows us to represent functions with new notation
- Representation allows for logic reduction
  - Implement same function with less logic
- Each square represents one minterm
- Each circle leads to one product term
- Not all functions can be reduced
- K-maps of four literals were considered (Larger examples exist)
- Don’t care conditions help minimize functions
- Result of minimization is a minimal sum-of-products
- Result contains prime implicants
- Essential prime implicants are required in the implementation